

Bypass de la Détection de Root Android

Instrumentation Dynamique avec Frida — Hooks Java et Natif

Lab 11 — Cours : Sécurité des Applications Mobiles

Application analysée :	OWASP UnCrackable Level 1
Package :	owasp.mstg.uncrackable1
Frida version :	16.0.19
Date :	3 mai 2026
Auditrice :	Chagdaly Hiba
Environnement :	Mobexler VM + Genymotion Android 10

Résumé

exécutif

Ce lab a porté sur le contournement des mécanismes de détection de root implémentés dans l'application OWASP UnCrackable Level 1. En combinant des hooks Java et natifs via Frida, les trois couches de protection de l'application ont été neutralisées avec succès, permettant à l'application de s'exécuter normalement sur un émulateur rooté sans déclencher d'alerte.

Contents

1	Comportement initial de l'application	2
2	Analyse des mécanismes de détection	2
3	Préparation et injection du bypass Java	3
4	Ajout des hooks natifs	4
5	Bypass ciblé de la classe de détection	4
6	Résultat final	5
7	Scripts utilisés	5
8	Conclusion	6

Comportement initial de l'application

Sans instrumentation, l'application UnCrackable Level 1 détecte immédiatement l'environnement rooté de l'émulateur Genymotion et affiche une alerte bloquante. L'application se ferme dès que l'utilisateur appuie sur OK, rendant toute interaction impossible.

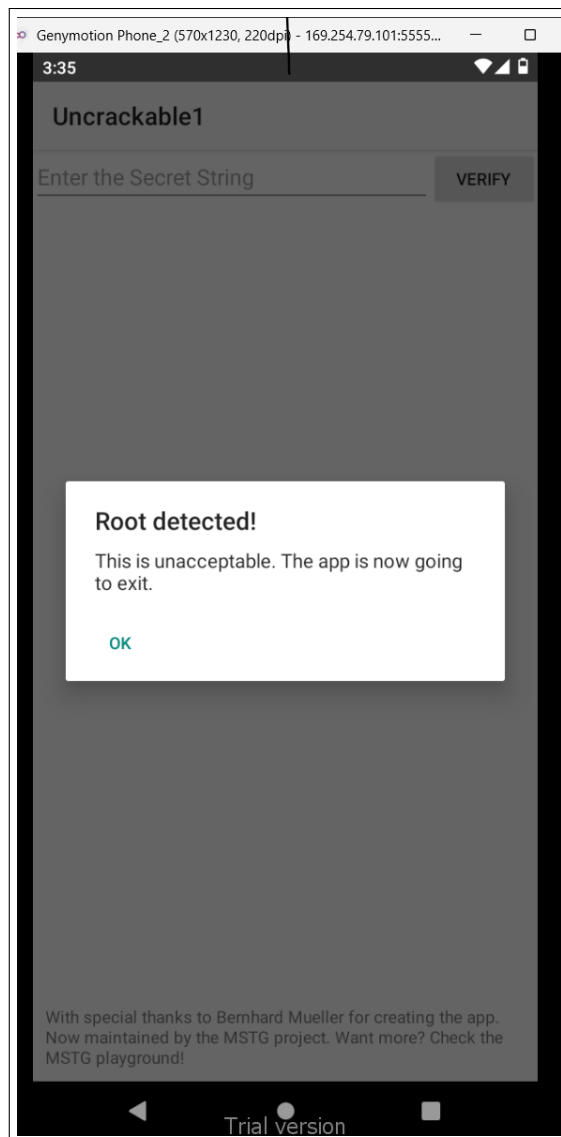


Figure 1: UnCrackable Level 1 — alerte Root detected sans bypass

Analyse des mécanismes de détection

Grâce à l'analyse statique réalisée dans le Lab 04, on sait qu'UnCrackable Level 1 implémente sa détection root dans la classe `sg.vantagepoint.a.c` avec trois méthodes distinctes. La méthode `c.a()` vérifie les tags de build, `c.b()` recherche des binaires suspects comme `su` et `busybox`, et `c.c()` vérifie les chemins système caractéristiques d'un appareil rooté. Cette connaissance du code source décompilé permet de cibler précisément les hooks nécessaires.

Préparation et injection du bypass Java

Le premier script de bypass a été créé pour neutraliser les vérifications Java. Il intercepte `File.exists()` pour retourner `false` sur les chemins suspects, force `Build.TAGS` à retourner `release-keys` au lieu de `test-keys`, et bloque les appels `Runtime.exec()` tentant d'exécuter `su` ou `busybox`.

```
mobexler@mobexler ~/Downloads
└─ cat > ~/Downloads/bypass_root.js << 'EOF'
Java.perform(function () {
  var RootPackages = ["com.noshufou.android.su", "com.thirdparty.superuser", "eu.chainfire.supersu"];
  var RootBinaries = ["/system/bin/su", "/system/xbin/su", "/sbin/su", "/system/app/Superuser.apk", "/system/xbin/daemonsu", "/system/bin/busybox"];

  var File = Java.use("java.io.File");
  File.exists.implementation = function() {
    var path = this.getAbsolutePath();
    if (RootBinaries.indexOf(path) >= 0) {
      console.log("[+] File.exists bypass: " + path);
      return false;
    }
    return this.exists.call(this);
  };

  var Build = Java.use("android.os.Build");
  Object.defineProperty(Build, "TAGS", { get: function() { return "release-keys"; } });
  console.log("[+] Build.TAGS -> release-keys");

  var Runtime = Java.use("java.lang.Runtime");
  Runtime.exec.overload("java.lang.String").implementation = function(cmd) {
    if (cmd.indexOf("su") >= 0 || cmd.indexOf("busybox") >= 0) {
      console.log("[+] Blocked Runtime.exec: " + cmd);
      return this.exec("echo");
    }
    return this.exec(cmd);
  };

  console.log("[+] Root bypass installed!");
});
EOF
```

Figure 2: Script `bypass_root.js` — hooks Java sur `File.exists`, `Build.TAGS` et `Runtime.exec`

L'injection du script produit les logs attendus confirmant que les hooks Java sont actifs. Les messages `[+] Build.TAGS -> release-keys` et les occurrences de `[+] File.exists bypass` attestent que la couche Java est correctement neutralisée.

```
mobexler@Mobexler ~/Downloads
└─ frida -U -f owasp.mstg.uncrackable1 -l ~/Downloads/bypass_root.js

Frida 16.0.19 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

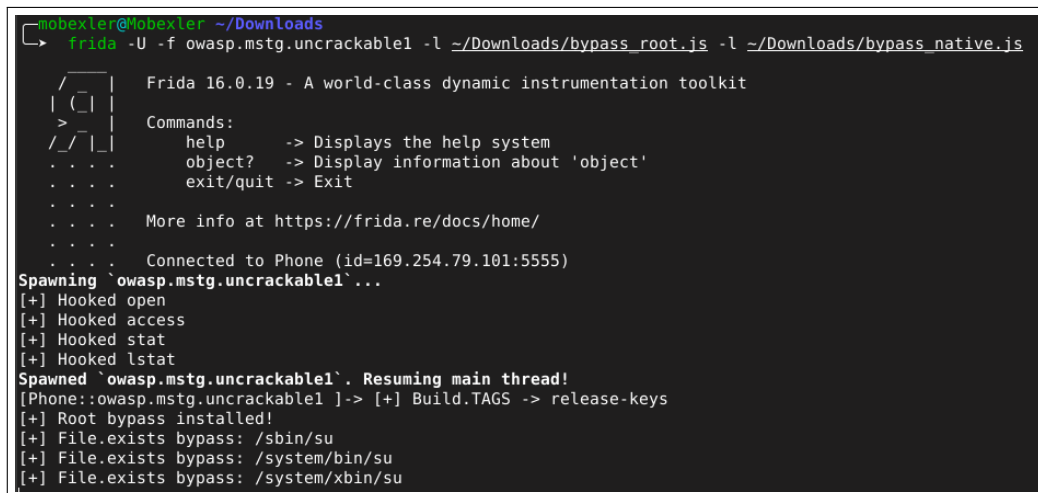
More info at https://frida.re/docs/home/

Connected to Phone (id=169.254.79.101:5555)
Spawned 'owasp.mstg.uncrackable1'. Resuming main thread!
[Phone::owasp.mstg.uncrackable1 ]-> [+] Build.TAGS -> release-keys
[+] Root bypass installed!
[+] File.exists bypass: /sbin/su
[+] File.exists bypass: /system/bin/su
[+] File.exists bypass: /system/xbin/su
[Phone::owasp.mstg.uncrackable1 ]-> %resume
[Phone::owasp.mstg.uncrackable1 ]-> |
```

Figure 3: Logs du bypass Java — `File.exists` et `Build.TAGS` interceptés avec succès

Ajout des hooks natifs

La détection Java seule ne suffit pas. UnCrackable Level 1 utilise également des appels natifs pour vérifier la présence de binaires suspects. Un second script `bypass_native.js` a été développé pour hooker les fonctions natives `open`, `access`, `stat` et `lstat`.



```
mobexier@mobexier ~/Downloads
└─$ frida -U -f owasp.mstg.uncrackable1 -l ~/Downloads/bypass_root.js -l ~/Downloads/bypass_native.js

Frida 16.0.19 - A world-class dynamic instrumentation toolkit

Commands:
  help           -> Displays the help system
  object?        -> Display information about 'object'
  exit/quit      -> Exit

More info at https://frida.re/docs/home/

Connected to Phone (id=169.254.79.101:5555)
Spawning `owasp.mstg.uncrackable1`...
[+] Hooked open
[+] Hooked access
[+] Hooked stat
[+] Hooked lstat
Spawned `owasp.mstg.uncrackable1`. Resuming main thread!
[Phone::owasp.mstg.uncrackable1 ]-> [+] Build.TAGS -> release-keys
[+] Root bypass installed!
[+] File.exists bypass: /sbin/su
[+] File.exists bypass: /system/bin/su
[+] File.exists bypass: /system/xbin/su
```

Figure 4: Hooks natifs actifs — `open`, `access`, `stat`, `lstat` interceptés avant le démarrage

Bypass ciblé de la classe de détection

Pour contourner définitivement la détection, un troisième script `bypass_uncrackable.js` a été développé en s'appuyant sur la connaissance du code source. Ce script cible directement les méthodes `c.a()`, `c.b()` et `c.c()` de la classe `sg.vantagepoint.a.c`.

L'injection produit les trois messages de confirmation, attestant que les trois vérifications de root sont neutralisées simultanément.

```

mobexler@Mobexler ~/Downloads
└─ cat > ~/Downloads/bypass_uncrackable.js << 'EOF'
Java.perform(function () {
  console.log("[+] Trying UnCrackable specific bypass...");

  try {
    var c = Java.use("sg.vantagepoint.a.c");
    c.a.implementation = function() {
      console.log("[+] Bypassed c.a() - root check 1");
      return false;
    };
    c.b.implementation = function() {
      console.log("[+] Bypassed c.b() - root check 2");
      return false;
    };
    c.c.implementation = function() {
      console.log("[+] Bypassed c.c() - root check 3");
      return false;
    };
    console.log("[+] UnCrackable bypass installed!");
  } catch(e) {
    console.log("[-] Error: " + e);
  }
});
EOF

```

Figure 5: Script `bypass_uncrackable.js` ciblant la classe `sg.vantagepoint.a.c`

```

mobexler@Mobexler ~/Downloads
└─ frida -U -f owasp.mstg.uncrackable1 -l ~/Downloads/bypass_uncrackable.js

Frida 16.0.19 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

More info at https://frida.re/docs/home/

Connected to Phone (id=169.254.79.101:5555)
Spawned `owasp.mstg.uncrackable1`. Resuming main thread!
[Phone::owasp.mstg.uncrackable1 ]-> [+] Trying UnCrackable specific bypass...
[+] UnCrackable bypass installed!
[+] Bypassed c.a() - root check 1
[+] Bypassed c.b() - root check 2
[+] Bypassed c.c() - root check 3

```

Figure 6: Bypass des méthodes `c.a()`, `c.b()`, `c.c()` — trois root checks neutralisés

Résultat final

Après l'injection du bypass ciblé, l'application UnCrackable Level 1 s'ouvre normalement sur l'émulateur rooté, sans aucune alerte. Le champ "Enter the Secret String" est accessible et le bouton VERIFY fonctionnel, démontrant le succès complet du contournement.

Scripts utilisés

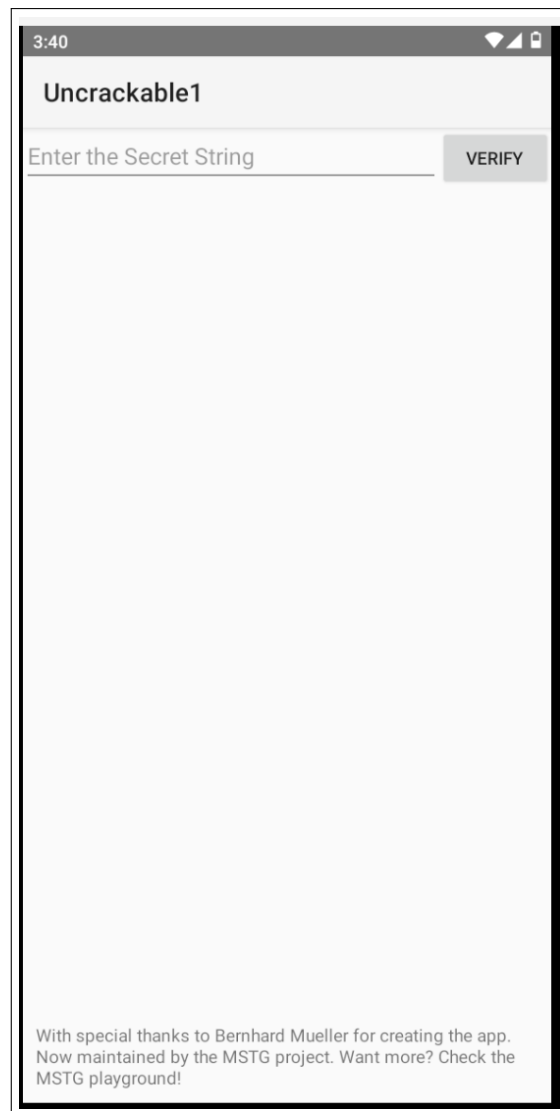


Figure 7: UnCrackable Level 1 — ouvert sans alerte après bypass complet

Script	Technique	Cible
bypass_root.js	Hook Java	File.exists, Build.TAGS, Runtime.exec
bypass_native.js	Hook natif	open, access, stat, lstat
bypass_uncrackable.js	Hook Java ciblé	sg.vantagepoint.a.c.a/b/c

Table 1: Scripts Frida utilisés pour le bypass

Conclusion

Ce lab démontre qu'une détection root multi-couches peut être contournée par instrumentation dynamique dès lors que le code source décompilé est connu. La combinaison d'une analyse statique préalable et d'une instrumentation Frida ciblée constitue une approche méthodique et efficace pour neutraliser les protections d'une application Android.

Rapport rédigé dans le cadre du cours de Sécurité des Applications Mobiles
Chagdaly Hiba — Mai 2026